

GUIDE D'INSTALLATION

PostgreSQL 16

Cluster Haute Disponibilité — Patroni + etcd + HAProxy + PgBouncer + pgBackRest

Composant	Version	Rôle
Ubuntu	24.04 LTS	Système d'exploitation de base
PostgreSQL	16.x	Moteur de base de données
Patroni	3.x	Orchestrateur de haute disponibilité
etcd	3.5.x	DCS — Consensus distribué
HAProxy	2.8.x LTS	Routeur TCP / Load Balancer
PgBouncer	1.22.x	Pooler de connexions
pgBackRest	2.50+	Sauvegarde et restauration PITR

Serveur	IP	Rôles installés
node1	10.0.0.1	PostgreSQL, Patroni, PgBouncer
node2	10.0.0.2	PostgreSQL, Patroni, PgBouncer
node3	10.0.0.3	PostgreSQL, Patroni, PgBouncer
etcd1	10.0.0.10	etcd
etcd2	10.0.0.11	etcd
etcd3	10.0.0.12	etcd
lb	10.0.0.20	HAProxy
backup-srv	10.0.0.30	pgBackRest + SFTP

0. Prérequis et préparation de l'environnement

Cette section est à exécuter sur TOUS les nœuds (node1, node2, node3, etcd1-3, lb, backup-srv) sauf mention contraire.

0.1 Mise à jour du système

ÉTAPE 1 — Mettre à jour Ubuntu 24.04

Sur tous les nœuds

```
apt update && apt upgrade -y
apt install -y curl gnupg2 wget lsb-release ca-certificates apt-transport-https
```

☑ Vérification

Vérification : `lsb_release -a` → Ubuntu 24.04 LTS

0.2 Configuration réseau et DNS

ÉTAPE 2 — Configurer `/etc/hosts` sur tous les nœuds

Pour la résolution de noms interne

```
cat >> /etc/hosts << 'EOF'
10.0.0.1    node1
10.0.0.2    node2
10.0.0.3    node3
10.0.0.10   etcd1
10.0.0.11   etcd2
10.0.0.12   etcd3
10.0.0.20   lb
10.0.0.30   backup-srv
EOF
```

0.3 Synchronisation NTP

ÉTAPE 3 — Activer et vérifier la synchronisation NTP

Essentiel pour `etcd` et `Patroni`

```
apt install -y chrony
systemctl enable chrony --now
chronyc tracking
# Vérifier : System time offset doit être < 0.1s
```

⚠ Attention

Une dérive de temps entre les nœuds peut provoquer des problèmes de quorum `etcd` et des comportements imprévisibles de `Patroni`.

0.4 Désactiver le swap

ÉTAPE 4 — Désactiver le swap

Recommandé pour `PostgreSQL` et `etcd`

```
swapoff -a
```

```
# Commenter la ligne swap dans /etc/fstab pour persistance
sed -i '/swap/s/^/#/' /etc/fstab
```

0.5 Paramètres kernel recommandés

ÉTAPE 5 — Configurer sysctl pour PostgreSQL

Sur les nœuds PostgreSQL uniquement

```
cat >> /etc/sysctl.d/99-postgresql.conf << 'EOF'
vm.overcommit_memory = 2
vm.overcommit_ratio = 80
vm.swappiness = 10
vm.dirty_ratio = 10
vm.dirty_background_ratio = 3
kernel.shmmax = 68719476736
EOF

sysctl --system
```

1. Installation et configuration d'etcd

À exécuter sur etcd1 (10.0.0.10), etcd2 (10.0.0.11) et etcd3 (10.0.0.12).

1.1 Installation

ÉTAPE 1 — Installer etcd

Sur etcd1, etcd2, etcd3

```
# Télécharger la dernière version stable d'etcd
ETCD_VER=v3.5.13
wget https://github.com/etcd-io/etcd/releases/download/${ETCD_VER}/etcd-
${ETCD_VER}-linux-amd64.tar.gz
tar xzf etcd-${ETCD_VER}-linux-amd64.tar.gz
mv etcd-${ETCD_VER}-linux-amd64/etcd* /usr/local/bin/

# Vérification
etcd --version
```

1.2 Configuration de chaque nœud etcd

ÉTAPE 2 — Créer l'utilisateur et les répertoires

Sur etcd1, etcd2, etcd3

```
useradd --system --no-create-home --shell /bin/false etcd
mkdir -p /var/lib/etcd /etc/etcd
chown -R etcd:etcd /var/lib/etcd /etc/etcd
```

ÉTAPE 3 — Créer le fichier de configuration — exemple pour etcd1

Adapter NODE_NAME et NODE_IP

```
# /etc/etcd/etcd.conf.yml (adapter pour etcd2 et etcd3)
cat > /etc/etcd/etcd.conf.yml << 'EOF'
name: etcd1
data-dir: /var/lib/etcd

listen-peer-urls: http://10.0.0.10:2380
listen-client-urls: http://10.0.0.10:2379,http://127.0.0.1:2379
advertise-client-urls: http://10.0.0.10:2379
initial-advertise-peer-urls: http://10.0.0.10:2380

initial-cluster:
etcd1=http://10.0.0.10:2380,etcd2=http://10.0.0.11:2380,etcd3=http://10.0.0.12:2380
initial-cluster-state: new
initial-cluster-token: pg-prod-cluster

heartbeat-interval: 100
election-timeout: 1000
EOF
```

i Information

Pour etcd2 : remplacer 10.0.0.10 par 10.0.0.11 et name: etcd1 par name: etcd2
 Pour etcd3 : remplacer 10.0.0.10 par 10.0.0.12 et name: etcd1 par name: etcd3

ÉTAPE 4 — Créer le service systemd etcd

Sur *etcd1*, *etcd2*, *etcd3*

```
cat > /etc/systemd/system/etcd.service << 'EOF'
[Unit]
Description=etcd key-value store
After=network.target

[Service]
User=etcd
ExecStart=/usr/local/bin/etcd --config-file /etc/etcd/etcd.conf.yml
Restart=always
RestartSec=5s
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable etcd --now
```

1.3 Vérification du cluster etcd

ÉTAPE 5 — Vérifier que le cluster etcd est sain

Depuis n'importe quel nœud

```
etcdctl --
endpoints=http://10.0.0.10:2379,http://10.0.0.11:2379,http://10.0.0.12:2379 \
  endpoint health
# Résultat attendu :
# http://10.0.0.10:2379 is healthy
# http://10.0.0.11:2379 is healthy
# http://10.0.0.12:2379 is healthy
```

```
# Vérifier l'élection du leader
etcdctl --endpoints=http://10.0.0.10:2379 endpoint status --write-out=table
```

☑ Vérification

3/3 nœuds etcd en état 'healthy' avant de continuer

Un leader etcd élu (IS LEADER = true sur exactement 1 nœud)

2. Installation de PostgreSQL 16

À exécuter sur node1, node2 et node3.

2.1 Ajout du dépôt officiel PGDG

ÉTAPE 1 — Ajouter le dépôt PostgreSQL Global Development Group

```
# Importer la clé GPG
curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc | \
  gpg --dearmor -o /etc/apt/trusted.gpg.d/postgresql.gpg

# Ajouter le dépôt
echo 'deb [signed-by=/etc/apt/trusted.gpg.d/postgresql.gpg] \
  https://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main' \
  > /etc/apt/sources.list.d/pgdg.list

apt update
```

ÉTAPE 2 — Installer PostgreSQL 16

```
apt install -y postgresql-16 postgresql-client-16 postgresql-contrib-16

# Arrêter le service PostgreSQL par défaut
# (Patroni gèrera le démarrage)
systemctl stop postgresql
systemctl disable postgresql

# Vérification de la version
/usr/lib/postgresql/16/bin/postgres --version
```

2.2 Initialisation du cluster PostgreSQL (node1 uniquement)

ÉTAPE 3 — Initialiser le cluster avec data_checksums

Sur node1 UNIQUEMENT — Patroni gèrera les autres

```
# Supprimer le cluster par défaut créé par apt
pg_dropcluster --stop 16 main

# Créer un nouveau cluster avec data_checksums et locale française
pg_createcluster --start 16 main -- \
  --data-checksums \
```

```
--locale=fr_FR.UTF-8 \
--encoding=UTF8 \
-D /var/lib/postgresql/16/main

# Vérifier data_checksums
sudo -u postgres psql -c 'SHOW data_checksums;'
# Doit retourner : on
```

⚠ Attention

data_checksums ne peut être activé qu'à la création du cluster (initdb).
 Sur un cluster existant, utiliser pg_checksums --enable (instance arrêtée).
 node2 et node3 seront initialisés automatiquement par Patroni via pg_basebackup.

2.3 Configuration de postgresql.conf

ÉTAPE 4 — Configurer les paramètres PostgreSQL

Sur node1 — sera propagé par Patroni aux autres

```
# Éditer /etc/postgresql/16/main/postgresql.conf

# Connexions
listen_addresses = '*'
port             = 5433
max_connections  = 200

# Mémoire (adapter selon la RAM disponible)
shared_buffers   = 8GB      # 25% de la RAM
effective_cache_size = 24GB  # 75% de la RAM
work_mem         = 64MB
maintenance_work_mem = 1GB

# WAL et réplication
wal_level        = replica
max_wal_senders  = 10
max_replication_slots = 10
wal_keep_size    = 256

# Archivage
archive_mode      = on
archive_command   = 'pgbackrest --stanza=main archive-push %p'
archive_timeout   = 300

# SSL
ssl               = on
ssl_cert_file     = '/etc/ssl/postgresql/server.crt'
ssl_key_file      = '/etc/ssl/postgresql/server.key'

# Logs
log_connections   = on
log_disconnections = on
log_replication_commands = on
log_line_prefix   = '%t [%p]: user=%u,db=%d,app=%a '

# Hot Standby
hot_standby       = on
max_standby_streaming_delay = 300s
hot_standby_feedback = on
```

2.4 Configuration de pg_hba.conf

ÉTAPE 5 — Configurer les règles d'authentification

Sur node1

```
# /etc/postgresql/16/main/pg_hba.conf

# Connexion locale postgres (administration)
local all postgres peer

# Connexions applicatives (adapter le sous-réseau)
hostssl all all 10.0.1.0/24 scram-sha-256

# Réplication physique entre nœuds
hostssl replication replicator 10.0.0.0/24 scram-sha-256

# PgBouncer (connexions poolées)
hostssl all all 10.0.0.0/24 scram-sha-256

# Monitoring
hostssl all monitoring 10.0.0.0/24 scram-sha-256
```

ÉTAPE 6 — Créer les rôles PostgreSQL

Sur node1

```
sudo -u postgres psql << 'EOF'
-- Rôle de réplication pour Patroni et les standbys
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'rep_password_fort';

-- Rôle applicatif
CREATE ROLE appuser WITH LOGIN PASSWORD 'app_password_fort';
CREATE DATABASE mabase OWNER appuser;

-- Rôle de monitoring (lecture seule)
CREATE ROLE monitoring WITH LOGIN PASSWORD 'mon_password_fort';
GRANT pg_monitor TO monitoring;
EOF
```

3. Installation et configuration de Patroni

À exécuter sur node1, node2 et node3.

3.1 Installation de Patroni

ÉTAPE 1 — Installer Patroni et ses dépendances

```
apt install -y python3 python3-pip python3-venv

# Installer Patroni avec support etcd3
pip3 install patroni[etcd3] psycopg2-binary --break-system-packages

# Vérification
patroni --version
```

3.2 Configuration de Patroni

ÉTAPE 2 — Créer le fichier de configuration Patroni

Adapter pour chaque nœud (name et listen)

```
# /etc/patroni.yml (exemple pour node1 – adapter IP et name pour node2/node3)

scope: pg-prod
namespace: /db/
name: node1

restapi:
  listen: 10.0.0.1:8008
  connect_address: 10.0.0.1:8008

etcd3:
  hosts:
    - 10.0.0.10:2379
    - 10.0.0.11:2379
    - 10.0.0.12:2379

bootstrap:
  dcs:
    ttl: 30
    loop_wait: 10
    retry_timeout: 10
    maximum_lag_on_failover: 1048576
    postgresql:
      use_pg_rewind: true
      use_slots: true
      parameters:
        wal_level: replica
        hot_standby: on
        max_wal_senders: 10
        max_replication_slots: 10
  initdb:
    - encoding: UTF8
    - data-checksums
  pg_hba:
    - hostssl replication replicator 10.0.0.0/24 scram-sha-256
    - hostssl all all 10.0.0.0/24 scram-sha-256
    - local all postgres peer

postgresql:
  listen: 10.0.0.1:5433
  connect_address: 10.0.0.1:5433
  data_dir: /var/lib/postgresql/16/main
  bin_dir: /usr/lib/postgresql/16/bin
  pgpass: /tmp/pgpass0
  authentication:
    replication:
      username: replicator
      password: rep_password_fort
    superuser:
      username: postgres
      password: pg_password_fort
  parameters:
    archive_mode: on
    archive_command: pgbackrest --stanza=main archive-push %p
    archive_timeout: 300
    shared_buffers: 8GB
```



```
work_mem: 64MB

tags:
  nofailover: false
  noloadbalance: false
  clonefrom: false
```

i Information

Pour node2 : name: node2, listen: 10.0.0.2:8008, connect_address: 10.0.0.2:8008

Pour node3 : name: node3, listen: 10.0.0.3:8008, connect_address: 10.0.0.3:8008

ÉTAPE 3 — Créer le service systemd Patroni

Sur node1, node2, node3

```
cat > /etc/systemd/system/patroni.service << 'EOF'
[Unit]
Description=Patroni High Availability PostgreSQL
After=syslog.target network.target

[Service]
Type=simple
User=postgres
Group=postgres
ExecStart=/usr/local/bin/patroni /etc/patroni.yml
ExecReload=/bin/kill -s HUP $MAINPID
KillMode=process
TimeoutSec=30
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable patroni
```

3.3 Démarrage du cluster Patroni

ÉTAPE 4 — Démarrer Patroni sur node1 en premier

node1 deviendra le primaire

```
# Sur node1 uniquement en premier
systemctl start patroni

# Suivre les logs
journalctl -u patroni -f
# Chercher : 'promoted self to leader'

# Vérifier l'état
patronictl -c /etc/patroni.yml list
```

ÉTAPE 5 — Démarrer Patroni sur node2 et node3

Après que node1 soit Leader

```
# Sur node2 et node3 (après que node1 soit 'Leader')
```

```
systemctl start patroni

# Patroni lance automatiquement un pg_basebackup depuis node1
# Suivre : journalctl -u patroni -f
# Attendre : 'started as a secondary'

# Vérification finale
patronictl -c /etc/patroni.yml list
```

☑ Vérification

patronictl list doit afficher :

```
node1 : Leader, state: running
node2 : Replica, state: streaming
node3 : Replica, state: streaming
Lag de réplication < 1s sur node2 et node3
```

4. Installation et configuration de HAProxy

À exécuter sur le serveur lb (10.0.0.20).

4.1 Installation

ÉTAPE 1 — Installer HAProxy 2.8 LTS

Sur lb uniquement

```
# Ajouter le dépôt HAProxy
add-apt-repository ppa:vbernat/haproxy-2.8 -y
apt update
apt install -y haproxy=2.8.*

# Vérification
haproxy -v
```

4.2 Configuration

ÉTAPE 2 — Configurer HAProxy pour PostgreSQL

Sur lb

```
# /etc/haproxy/haproxy.cfg
global
    maxconn 500
    log /dev/log local0
    log /dev/log local1 notice

defaults
    mode tcp
    log global
    option tcplog
    timeout connect 5s
    timeout client 30s
    timeout server 30s
    retries 3
```

```
# — Dashboard statistiques
listen stats
    bind *:7000
    mode http
    stats enable
    stats uri /
    stats refresh 5s
    stats show-legends

# — Frontend ÉCRITURE — Port 5432 → Primaire
frontend pg_write
    bind *:5432
    default_backend primary_backend

backend primary_backend
    option httpchk GET /primary
    http-check expect status 200
    default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
    server node1 10.0.0.1:6432 check port 8008
    server node2 10.0.0.2:6432 check port 8008
    server node3 10.0.0.3:6432 check port 8008

# — Frontend LECTURE — Port 5433 → Standbys
frontend pg_read
    bind *:5433
    default_backend replica_backend

backend replica_backend
    balance roundrobin
    option httpchk GET /replica
    http-check expect status 200
    default-server inter 3s fall 3 rise 2
    server node2 10.0.0.2:6432 check port 8008
    server node3 10.0.0.3:6432 check port 8008
    server node1 10.0.0.1:6432 check port 8008 backup
```

ÉTAPE 3 — Démarrer HAProxy

```
haproxy -c -f /etc/haproxy/haproxy.cfg # Vérifier la config
systemctl enable haproxy --now

# Vérifier le dashboard
curl -s http://10.0.0.20:7000/ | grep -i 'node[123]'
```

☑ Vérification

Dashboard HAProxy accessible sur <http://10.0.0.20:7000/>
 node1 en vert sur le backend primary_backend
 node2 et node3 en vert sur replica_backend

5. Installation et configuration de PgBouncer

À exécuter sur node1, node2 et node3.

5.1 Installation

ÉTAPE 1 — Installer PgBouncer

Sur *node1*, *node2*, *node3*

```
apt install -y pgbouncer

# Vérification
pgbouncer --version
```

5.2 Configuration

ÉTAPE 2 — Configurer pgbouncer.ini

Sur *chaque nœud* — pointer vers le PostgreSQL local

```
# /etc/pgbouncer/pgbouncer.ini

[databases]
# Pointer vers le PostgreSQL local du nœud
mabase = host=127.0.0.1 port=5433 dbname=mabase
* = host=127.0.0.1 port=5433 # catch-all pour toutes les bases

[pgbouncer]
pool_mode      = transaction
max_client_conn = 1000
default_pool_size = 20
min_pool_size   = 5
reserve_pool_size = 5
reserve_pool_timeout = 3

listen_addr     = *
listen_port     = 6432

auth_type       = scram-sha-256
auth_file       = /etc/pgbouncer/userlist.txt

# Timeouts
connect_timeout = 15
server_idle_timeout = 600

# Logging
log_connections  = 1
log_disconnections = 1
log_pooler_errors = 1

# Pidfile
pidfile          = /var/run/postgresql/pgbouncer.pid
logfile          = /var/log/postgresql/pgbouncer.log
```

ÉTAPE 3 — Créer le fichier d'authentification userlist.txt

```
# Générer les hash SCRAM-SHA-256 pour les utilisateurs
sudo -u postgres psql -t -c \
  "SELECT '\"' || rolname || '\"' || rolpassword || '\"'
  FROM pg_authid WHERE rolpassword IS NOT NULL;" \
  >> /etc/pgbouncer/userlist.txt

# Ajuster les permissions
chown postgres:postgres /etc/pgbouncer/userlist.txt
```

```
chmod 640 /etc/pgbouncer/userlist.txt
```

ÉTAPE 4 — Démarrer PgBouncer

```
systemctl enable pgbouncer --now

# Vérifier les pools
psql -h 127.0.0.1 -p 6432 -U pgbouncer pgbouncer -c 'SHOW POOLS;'
```

☑ Vérification

PgBouncer écoute sur le port 6432

SHOW POOLS affiche les bases configurées avec `cl_active = 0` (pas encore de clients)

6. Installation et configuration de pgBackRest (SFTP)

6.1 Préparation du serveur de sauvegarde (backup-srv)

ÉTAPE 1 — Créer l'utilisateur et les répertoires

Sur backup-srv uniquement

```
# Créer l'utilisateur dédié (sans shell interactif)
useradd --system --create-home --shell /usr/sbin/nologin pgbackrest

# Créer le répertoire de stockage
mkdir -p /data/pgbackrest
chown -R pgbackrest:pgbackrest /data/pgbackrest
chmod 750 /data/pgbackrest

# Créer le répertoire .ssh
mkdir -p /home/pgbackrest/.ssh
chown pgbackrest:pgbackrest /home/pgbackrest/.ssh
chmod 700 /home/pgbackrest/.ssh
touch /home/pgbackrest/.ssh/authorized_keys
chown pgbackrest:pgbackrest /home/pgbackrest/.ssh/authorized_keys
chmod 600 /home/pgbackrest/.ssh/authorized_keys

# Vérifier que le sousystème SFTP est actif
grep 'Subsystem sftp' /etc/ssh/sshd_config
# Si absent : echo 'Subsystem sftp /usr/lib/openssh/sftp-server' >>
/etc/ssh/sshd_config
# systemctl restart sshd
```

6.2 Installation de pgBackRest sur les nœuds PostgreSQL

ÉTAPE 2 — Installer pgBackRest >= 2.47

Sur node1, node2, node3

```
apt install -y pgbackrest

# Vérifier la version (doit être >= 2.47 pour le support SFTP natif)
```

```
pgbackrest version

# Créer le répertoire de config et logs
mkdir -p /etc/pgbackrest /var/log/pgbackrest
chown postgres:postgres /etc/pgbackrest /var/log/pgbackrest
```

6.3 Génération des clés SSH

ÉTAPE 3 — Générer une paire de clés ED25519 sur chaque nœud

Sur *node1*, *node2*, *node3*

```
# Sur chaque nœud PostgreSQL – générer la clé SSH
mkdir -p /etc/pgbackrest/.ssh
chown postgres:postgres /etc/pgbackrest/.ssh
chmod 700 /etc/pgbackrest/.ssh

sudo -u postgres ssh-keygen -t ed25519 \
  -f /etc/pgbackrest/.ssh/id_ed25519 \
  -N '' \
  -C 'pgbackrest@$(hostname)'

# Afficher la clé publique (à copier sur backup-srv)
cat /etc/pgbackrest/.ssh/id_ed25519.pub
```

ÉTAPE 4 — Copier les clés publiques vers backup-srv

Pour chaque nœud

```
# Depuis chaque nœud (node1, node2, node3)
# Option 1 : ssh-copy-id (si connexion SSH disponible)
ssh-copy-id -i /etc/pgbackrest/.ssh/id_ed25519.pub pgbackrest@10.0.0.30

# Option 2 : copie manuelle
# Sur backup-srv, ajouter la clé publique :
# echo '<contenu_id_ed25519.pub>' >> /home/pgbackrest/.ssh/authorized_keys
```

ÉTAPE 5 — Récupérer l'empreinte du serveur SFTP

Depuis n'importe quel nœud PostgreSQL

```
# Obtenir l'empreinte SHA256 du serveur backup-srv
ssh-keyscan -t ed25519 10.0.0.30 | ssh-keygen -lf -
# Exemple : 256 SHA256:AbCdEfGhIjKlMnOpQrSt... 10.0.0.30 (ED25519)

# Copier la valeur SHA256:... pour la configuration pgbackrest.conf
```

6.4 Configuration de pgbackrest.conf

ÉTAPE 6 — Créer /etc/pgbackrest/pgbackrest.conf

Sur *node1*, *node2*, *node3* — identique

```
cat > /etc/pgbackrest/pgbackrest.conf << 'EOF'
[global]

# — Dépôt SFTP
```

```

repo1-type                = sftp
repo1-path                = /data/pgbackrest/pg-prod
repo1-sftp-host           = 10.0.0.30
repo1-sftp-host-user      = pgbackrest
repo1-sftp-host-key-hash-type = sha256
repo1-sftp-private-key-file = /etc/pgbackrest/.ssh/id_ed25519
repo1-sftp-public-key-file  = /etc/pgbackrest/.ssh/id_ed25519.pub

# — Chiffrement (obligatoire)
repo1-cipher-type         = aes-256-cbc
repo1-cipher-pass         = <passphrase_tres_forte>

# — Rétention
repo1-retention-full      = 4
repo1-retention-diff      = 6

# — Performance
process-max               = 4
compress-type             = zst
compress-level            = 3

[main]
pg1-path = /var/lib/postgresql/16/main
pg1-port = 5433
pg1-user = postgres
EOF

chown postgres:postgres /etc/pgbackrest/pgbackrest.conf
chmod 640 /etc/pgbackrest/pgbackrest.conf

```

6.5 Initialisation et première sauvegarde

ÉTAPE 7 — Initialiser la stanza et effectuer la première sauvegarde

Sur *node1*

```

# Créer la structure de répertoires sur le serveur SFTP
sudo -u postgres pgbackrest --stanza=main --log-level-console=info stanza-
create

# Vérifier la configuration complète
sudo -u postgres pgbackrest --stanza=main --log-level-console=detail check
# Résultat attendu : check command end: completed successfully

# Lancer la première sauvegarde complète
sudo -u postgres pgbackrest --stanza=main --type=full \
  --log-level-console=info backup

# Vérifier la sauvegarde
sudo -u postgres pgbackrest --stanza=main info
# status: ok — label affiché

```

ÉTAPE 8 — Configurer les sauvegardes automatiques (cron)

Sur *node1*

```

# Ajouter au crontab de l'utilisateur postgres
sudo -u postgres crontab -e

# Ajouter les lignes suivantes :

```

```
# Sauvegarde complète le dimanche à 02h00
0 2 * * 0 pgbackrest --stanza=main --type=full backup

# Sauvegarde différentielle le vendredi à 02h00
0 2 * * 5 pgbackrest --stanza=main --type=diff backup

# Sauvegarde incrémentielle du lundi au samedi (sauf vendredi) à 03h00
0 3 * * 1-4,6 pgbackrest --stanza=main --type=incr backup
```

☑ Vérification

pgbackrest info affiche status: ok avec une sauvegarde full listée

Vérifier les fichiers sur backup-srv : ls /data/pgbackrest/pg-prod/backup/

L'archivage WAL fonctionne : archive_command testée avec pgbackrest check

7. Génération des certificats SSL

À exécuter pour sécuriser les connexions de réplication et applicatives.

7.1 Créer une autorité de certification (CA) interne

ÉTAPE 1 — Générer la CA racine

Sur un poste d'administration sécurisé

```
mkdir -p /etc/ssl/postgresql/{ca,server,client}
cd /etc/ssl/postgresql

# Générer la clé privée CA
openssl genrsa -aes256 -out ca/root.key 4096

# Générer le certificat CA auto-signé (10 ans)
openssl req -new -x509 -days 3650 -key ca/root.key \
  -out ca/root.crt \
  -subj '/CN=PostgreSQL-CA/O=MonEntreprise/C=FR'
```

7.2 Créer les certificats serveur

ÉTAPE 2 — Générer un certificat pour chaque nœud PostgreSQL

Répéter pour node1, node2, node3

```
# Générer la clé et le CSR pour node1
openssl genrsa -out server/node1.key 2048
openssl req -new -key server/node1.key \
  -out server/node1.csr \
  -subj '/CN=node1/O=MonEntreprise/C=FR'

# Signer le certificat avec la CA
openssl x509 -req -days 1825 \
  -in server/node1.csr \
  -CA ca/root.crt -CAkey ca/root.key -CAcreateserial \
  -out server/node1.crt

# Copier sur le nœud
cp server/node1.crt /etc/ssl/postgresql/server.crt
```



```
cp server/node1.key /etc/ssl/postgresql/server.key
cp ca/root.crt /etc/ssl/postgresql/root.crt
chown postgres:postgres /etc/ssl/postgresql/*.{crt,key}
chmod 600 /etc/ssl/postgresql/server.key
```

8. Vérification finale du cluster

8.1 Tests de connectivité

ÉTAPE 1 — Tester la connexion en écriture via HAProxy

```
# Test connexion écriture (primaire)
psql -h 10.0.0.20 -p 5432 -U appuser -d mabase \
  -c 'SELECT pg_is_in_recovery();'
# Doit retourner : f (false = primaire)

# Test connexion lecture (standby)
psql -h 10.0.0.20 -p 5433 -U appuser -d mabase \
  -c 'SELECT pg_is_in_recovery();'
# Doit retourner : t (true = standby)
```

ÉTAPE 2 — Vérifier la réplication

```
# Sur le primaire – état de tous les standbys
psql -h 10.0.0.20 -p 5432 -U postgres \
  -c 'SELECT application_name, state, replay_lag FROM pg_stat_replication;'
# Doit afficher node2 et node3 en state = streaming

# Lag de réplication
psql -h 10.0.0.20 -p 5432 -U postgres \
  -c "SELECT application_name, replay_lag FROM pg_stat_replication;"
# Lag doit être < 1 seconde
```

ÉTAPE 3 — Simuler un failover Patroni

```
# Vérifier l'état initial
patronictl -c /etc/patroni.yml list

# Déclencher un switchover planifié
patronictl -c /etc/patroni.yml switchover pg-prod \
  --master node1 --candidate node2 --scheduled now

# Attendre 10 secondes puis vérifier
patronictl -c /etc/patroni.yml list
# node2 doit être Leader, node1 et node3 en Replica

# Remettre node1 en leader (optionnel)
patronictl -c /etc/patroni.yml switchover pg-prod \
  --master node2 --candidate node1 --scheduled now
```

8.2 Checklist de validation finale

#	Vérification	Commande	Statut
1	etcd : 3/3 nœuds healthy	etcdctl endpoint health	☐
2	Patroni : Leader + 2 Replica	patronictl list	☐
3	Lag réplication < 1s	SELECT replay_lag FROM pg_stat_replication	☐
4	HAProxy : dashboard accessible	curl http://10.0.0.20:7000/	☐
5	PgBouncer : pools actifs	psql -p 6432 pgbouncer -c 'SHOW POOLS'	☐
6	pgBackRest : check OK	pgbackrest --stanza=main check	☐
7	Première sauvegarde full OK	pgbackrest --stanza=main info	☐
8	SSL : connexions chiffrées	psql ... -c 'SELECT ssl FROM pg_stat_ssl'	☐
9	data_checksums activé	SHOW data_checksums → on	☐
10	Switchover planifié réussi	patronictl switchover	☐

Bonne pratique

Ne passer en production qu'une fois toutes les cases cochées.
Documenter les RTO et RPO mesurés lors des tests dans le document d'architecture.

9. Dépannage — Erreurs fréquentes

Erreur / Symptôme	Cause probable	Solution
Patroni ne démarre pas (YAML error)	Erreur de syntaxe dans patroni.yml	Valider avec : <code>python3 -c 'import yaml; yaml.safe_load(open("/etc/patroni.yml"))'</code>
etcd : no leader elected	Moins de 2 nœuds etcd joignables	Vérifier réseau + firewall port 2380 entre les nœuds etcd
pg_basebackup échec (authentication failed)	pg_hba.conf manque l'entrée replication	Vérifier hostssl replication replicator 10.0.0.0/24 scram-sha-256
pgBackRest SFTP : Permission denied	Clé SSH non copiée sur backup-srv	Vérifier /home/pgbackrest/.ssh/authorized_keys
pgBackRest SFTP : Host key verification failed	Empreinte non configurée	Récupérer avec ssh-keyscan + ajouter repo1-sftp-host-key
HAProxy : tous	Patroni API inaccessible	Vérifier systemctl status patroni + port 8008

Erreur / Symptôme	Cause probable	Solution
backends DOWN		ouvert
Slot de réplication bloqué	Standby déconnecté longtemps	pg_drop_replication_slot si orphelin — attention perte de position
Conflit Hot Standby fréquent	max_standby_streaming_delay trop court	Augmenter à 300s ou activer hot_standby_feedback = on
archive_command échoue	Connexion SFTP interrompue	Tester : sudo -u postgres pgbackrest --stanza=main check
PgBouncer : auth error	userlist.txt périmé	Régénérer depuis pg_authid et recharger pgbouncer

Annexe — Commandes de référence rapide

Patroni

```

patronictl -c /etc/patroni.yml list                # État du cluster
patronictl -c /etc/patroni.yml switchover pg-prod   # Bascule planifiée
patronictl -c /etc/patroni.yml failover pg-prod     # Failover forcé
patronictl -c /etc/patroni.yml reinit pg-prod node1 # Reconstruire un nœud
patronictl -c /etc/patroni.yml edit-config          # Modifier la config
(tous les nœuds)
patronictl -c /etc/patroni.yml pause                # Suspendre Patroni
(maintenance)
patronictl -c /etc/patroni.yml resume               # Reprendre Patroni

```

pgBackRest

```

pgbackrest --stanza=main info                      # Lister les
sauvegardes
pgbackrest --stanza=main check                      # Vérifier la config
pgbackrest --stanza=main --type=full backup         # Sauvegarde complète
pgbackrest --stanza=main --type=incr backup         # Sauvegarde
incrémentielle
pgbackrest --stanza=main restore --delta            # Restauration delta
pgbackrest --stanza=main restore \
  --target='2024-03-15 14:31:00' \
  --target-action=promote                           # PITR à une heure
précise

```

PostgreSQL — Réplication

```

-- Lag de réplication (primaire)
SELECT application_name, state, replay_lag FROM pg_stat_replication;

-- WAL retenu par les slots
SELECT slot_name, active, pg_size_pretty(
  pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS wal_retenu
FROM pg_replication_slots;

```

```
-- Conflits Hot Standby (standby)
SELECT datname, confl_snapshot, confl_lock FROM pg_stat_database_conflicts;

-- Lag applicatif (standby)
SELECT now() - pg_last_xact_replay_timestamp() AS lag;
```